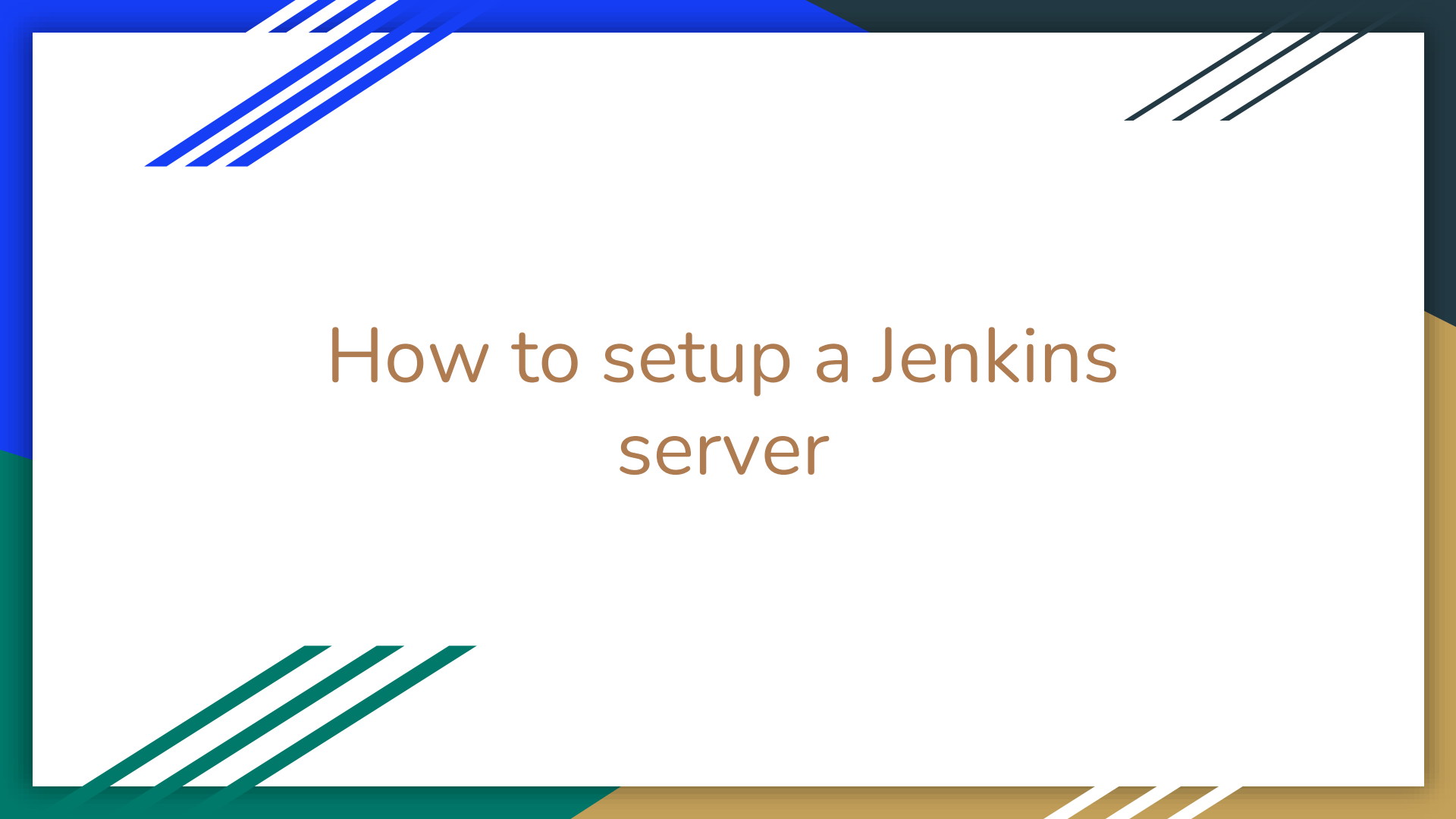




How to setup a Jenkins server

What is Jenkins

Jenkins

Build great things at any scale

The leading open source automation server, Jenkins provides hundreds of plugins to support building, deploying and automating any project.

The Big IDEA

Get something done automatically (hands-free) that involves:

- Multiple Steps
- Steps have dependencies
- May have to execute multiple things on multiple servers

One Example of What You Can do with Jenkins

- At 3:21 on Sunday
- Get token from Vault using predefined authentication variables (inserted by OPS)
- Login to Google Cloud using that token
- Copy 2TB of data from HDFS
- Process that data using a Python program
- Copy output to Google Dataflow
- If it fails or takes more than 60 minutes, page on-call using Pagerduty plugin
- If it succeeds, send our team an email.
- Delete the token, because security does not allow tokens to live outside Vault.
- Keep detailed logs for 30 last runs.
- Keep other metadata like run-times, also.

Another Example

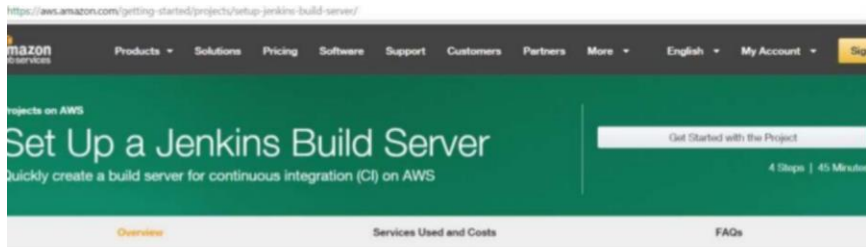
- Grab new version of code from git repo
- Integrate it with maven or some other tool
- Deploy it to 500 servers
- Do some automated testing
- Decided if this new version is good or not
- If automated testing fails, roll the whole thing back and page on-call

There are Other Comparable Tools

	Jenkins	Bamboo	Travis
Open Source	Yes	No	Yes
Plugins	Yes (Lots of them)	Expensive	Few
Support	Yes (Being Open Source helps)	Not Much (Small user community)	No
Easy Usage	No (The plugins help)	Yes	Yes
Atlassian Product Integration	Yes	Yes	Yes
Repo Support	All	All	Github

You Can Use Jenkins on AWS

(if you don't feel like managing your own)



In this project, you will learn how to deploy and host Jenkins, an open-source automation software predominantly used for CI/CD (Continuous Integration/Continuous Deployment). You will use Amazon Elastic Compute Cloud (EC2) in a public subnet within your own Amazon Virtual Private Cloud (VPC) and you will also set up an Amazon Elastic Block Store (EBS) volume.

What you'll accomplish:

Launch a virtual application server to host your Jenkins installation using [Amazon EC2](#). In this single-node setup, both the Jenkins server and agent run on the same Amazon EC2 instance. The instance type you provision will match your expected load. You will also be able to resize the instance if your expected load changes.

Deploy Jenkins on Amazon EC2. You will download and deploy the latest version of Jenkins.

Configure Jenkins with build servers. You will configure Jenkins so that it spins up additional Amazon EC2



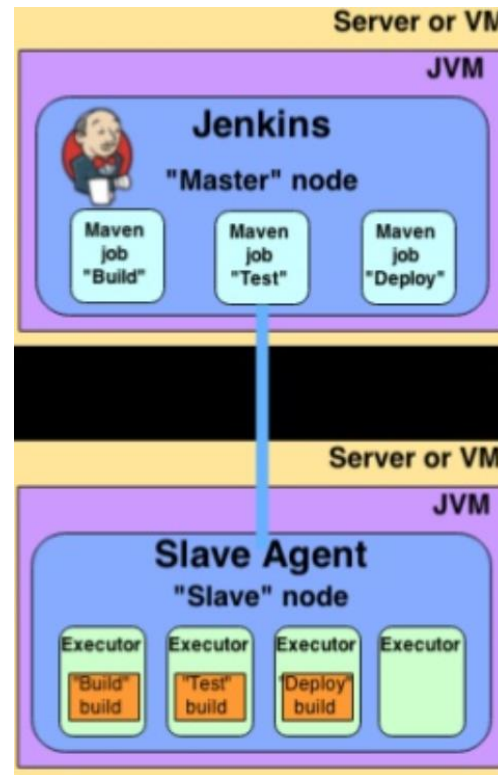
Installation

Java, Jenkins



Jenkins Architecture

- You can have many slaves
- Each slave can have labels
- Nothing except java is needed on slaves
- Just a jar file that the master pushes during agent installation



Login to Jenkins via your Favorite Browser



The image shows the Jenkins login page. At the top, there is a black header bar with the Jenkins logo (a cartoon character) on the left, the word "Jenkins" in white text in the center, and a "log In" link in white text on the right. Below the header, there is a light green horizontal bar with the word "Jenkins" and a right-pointing arrow. The main content area is white and contains the login form. The form has two labels, "User:" and "Password:", each followed by a text input field. Below the password field is a checkbox labeled "Remember me on this computer". At the bottom left of the form is a blue button with the text "log In" in white.

Jenkins log In

Jenkins

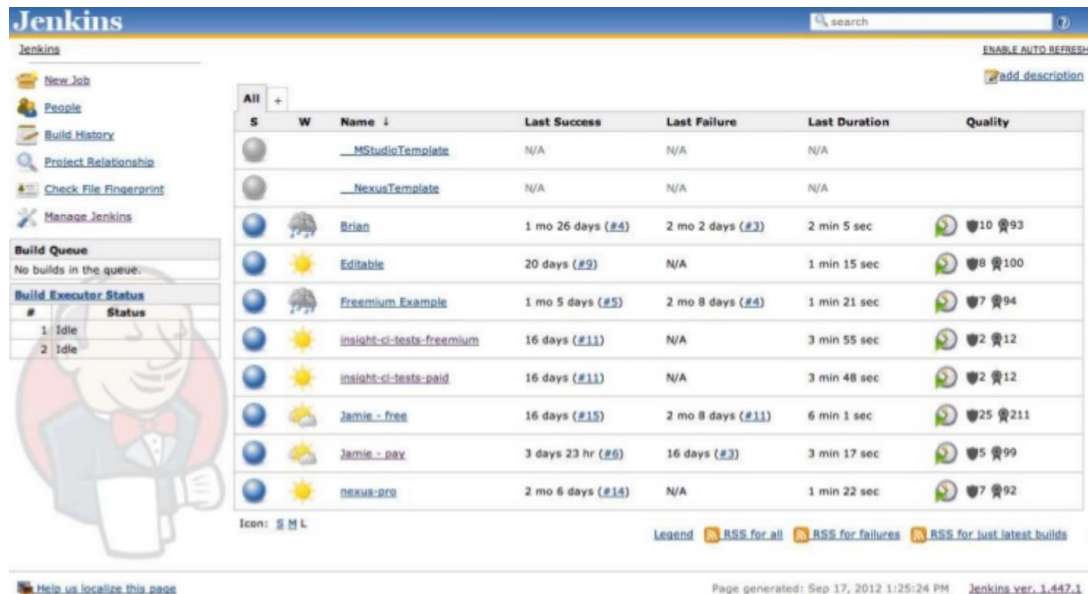
User:

Password:

☐ Remember me on this computer

log In

First Page You See (Your Mileage May Vary)



The screenshot shows the Jenkins web interface. The top navigation bar includes links for [New Job](#), [People](#), [Build History](#), [Project Relationship](#), [Check File Fingerprint](#), and [Manage Jenkins](#). The main content area displays a table of jobs with columns for Status (S), Name, Last Success, Last Failure, Last Duration, and Quality. A sidebar on the left shows the 'Build Queue' (empty) and 'Build Executor Status' (2 idle executors). A large cartoon character of a man in a tuxedo is overlaid on the bottom left. The footer contains a 'Help us localize this page' link, the page generation timestamp 'Page generated: Sep 17, 2012 1:25:24 PM', and the version 'Jenkins ver. 1.447.1'.

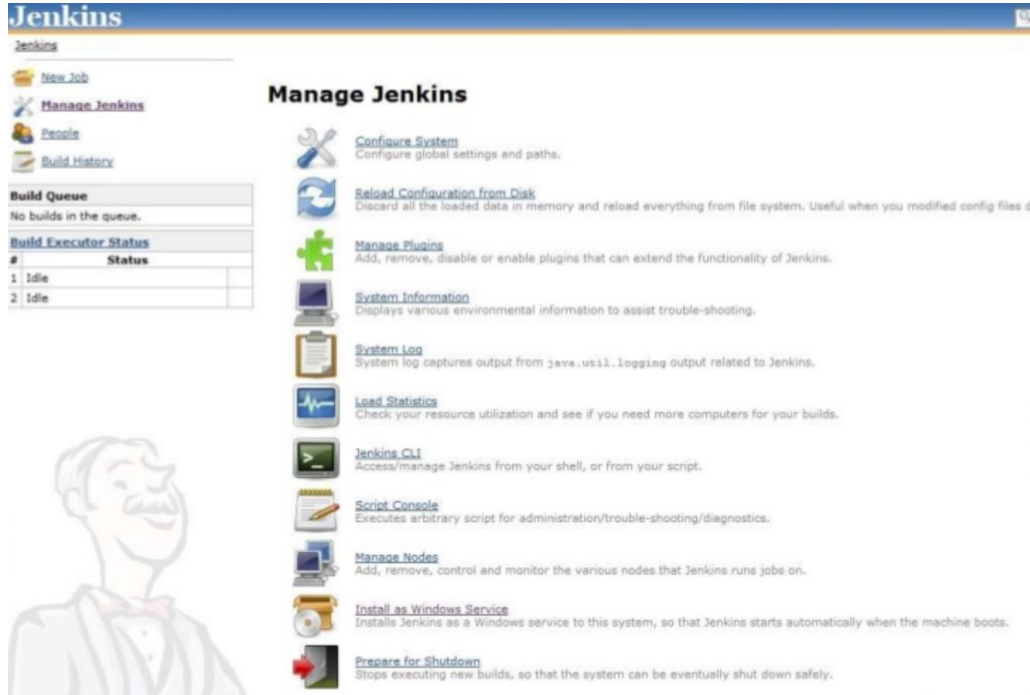
S	W	Name	Last Success	Last Failure	Last Duration	Quality
		_MStudioTemplate	N/A	N/A	N/A	
		_NexusTemplate	N/A	N/A	N/A	
		Brian	1 mo 26 days (#4)	2 mo 2 days (#3)	2 min 5 sec	10 93
		Editable	20 days (#9)	N/A	1 min 15 sec	8 100
		Freemium Example	1 mo 5 days (#5)	2 mo 8 days (#4)	1 min 21 sec	7 94
		insight-ci-tests-freemium	16 days (#11)	N/A	3 min 55 sec	2 12
		insight-ci-tests-paid	16 days (#11)	N/A	3 min 48 sec	2 12
		Jamie - free	16 days (#15)	2 mo 8 days (#11)	6 min 1 sec	25 211
		Jamie - pay	3 days 23 hr (#6)	16 days (#3)	3 min 17 sec	5 99
		nexus-pro	2 mo 6 days (#14)	N/A	1 min 22 sec	7 92

Icon: S M L

Legend RSS for all RSS for failures RSS for just latest builds

[Help us localize this page](#) Page generated: Sep 17, 2012 1:25:24 PM Jenkins ver. 1.447.1

Manage Jenkins Screen



The image shows the Jenkins web interface. At the top is the Jenkins logo and a navigation bar. On the left is a sidebar with links: New Job, Manage Jenkins, People, and Build History. Below these is a 'Build Queue' section showing 'No builds in the queue.' and a 'Build Executor Status' table. The main area is titled 'Manage Jenkins' and contains a list of administrative links, each with an icon and a description. A faint cartoon character is visible in the bottom left corner.

Jenkins

[New Job](#)

[Manage Jenkins](#)

[People](#)

[Build History](#)

Build Queue

No builds in the queue.

Build Executor Status

#	Status
1	Idle
2	Idle

Manage Jenkins

-  [Configure System](#)
Configure global settings and paths.
-  [Reload Configuration from Disk](#)
Discard all the loaded data in memory and reload everything from file system. Useful when you modified config files d
-  [Manage Plugins](#)
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.
-  [System Information](#)
Displays various environmental information to assist trouble-shooting.
-  [System Log](#)
System log captures output from `java.util.logging` output related to Jenkins.
-  [Load Statistics](#)
Check your resource utilization and see if you need more computers for your builds.
-  [Jenkins CLI](#)
Access/manage Jenkins from your shell, or from your script.
-  [Script Console](#)
Executes arbitrary script for administration/trouble-shooting/diagnostics.
-  [Manage Nodes](#)
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.
-  [Install as Windows Service](#)
Installs Jenkins as a Windows service to this system, so that Jenkins starts automatically when the machine boots.
-  [Prepare for Shutdown](#)
Stops executing new builds, so that the system can be eventually shut down safely.

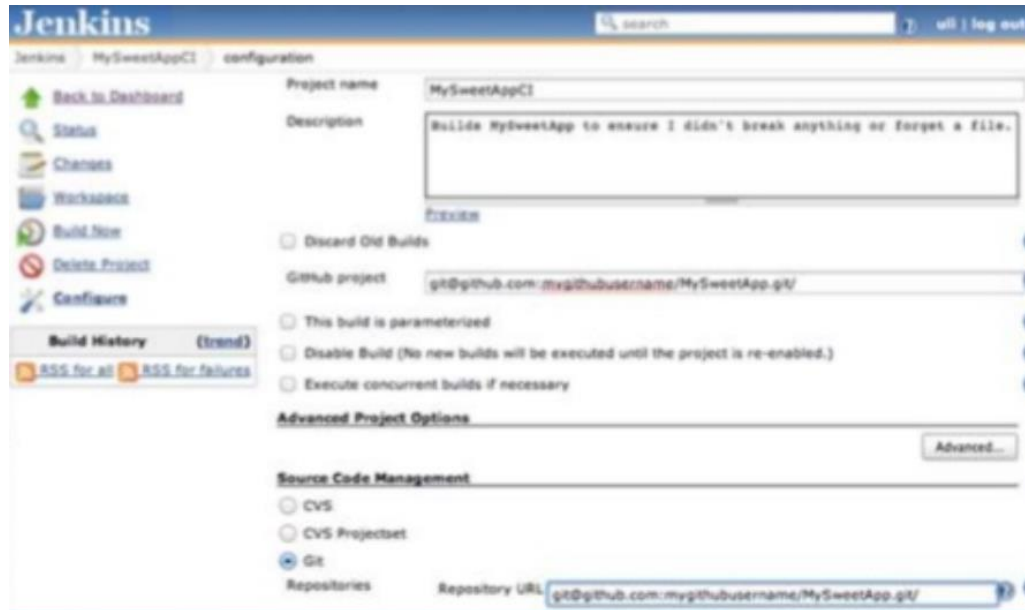
Build a New Job (Project)

The image shows the Jenkins 'New Item' configuration page. Three red arrows indicate the steps to create a new job:

- 1**: Points to the 'Item name' text box, which contains 'testproj1'.
- 2**: Points to the 'Freestyle project' radio button, which is selected.
- 3**: Points to the 'OK' button at the bottom right of the form.

The form includes a sidebar with navigation links: New Item, People, Build History, Manage Jenkins, and Credentials. The main content area shows the 'Freestyle project' selected, with a description: 'This is the central feature of Jenkins, allowing you to build software with any build system or SCM with any build software build.' Other options include 'Maven project', 'External Job', and 'Multi-configuration project'. The 'Build Queue' section shows 'No builds in the queue.' and the 'Build Executor Status' section shows two idle executors.

Build a New Job



The screenshot shows the Jenkins web interface for configuring a new job. The top navigation bar includes the Jenkins logo, a search bar, and user information (ull | log out). The breadcrumb trail indicates the current path: Jenkins > MySweetAppCI > configuration.

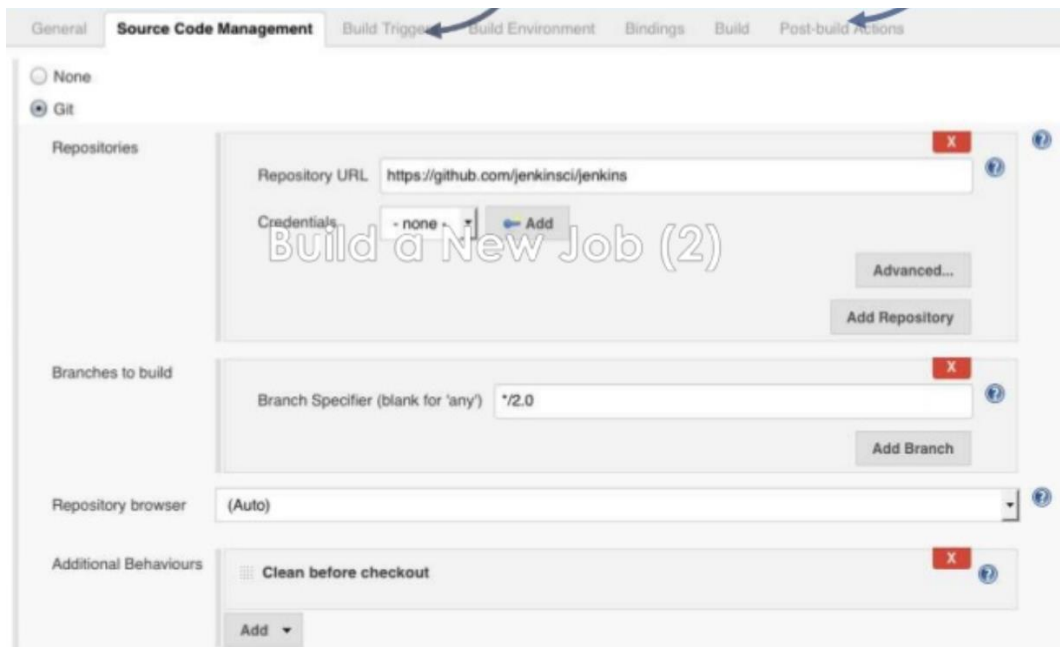
Left Sidebar:

- Back to Dashboard
- Status
- Changes
- Workspaces
- Build Now
- Delete Project
- Configure
- Build History (trend)**
- RSS for all
- RSS for failures

Main Configuration Area:

- Project name:** MySweetAppCI
- Description:** Builde MySweetApp to ensure I didn't break anything or forget a file.
- Preview** button
- ☐ Discard Old Builds
- Git project:** git@github.com:mygithubusername/MySweetApp.git/
- ☐ This build is parameterized
- ☐ Disable Build (No new builds will be executed until the project is re-enabled.)
- ☐ Execute concurrent builds if necessary
- Advanced Project Options** (with an **Advanced...** button)
- Source Code Management:**
 - ☐ CVS
 - ☐ CVS Projectset
 - ☒ Git
- Repositories:** Repository URL: git@github.com:mygithubusername/MySweetApp.git/

Build a New Job



The screenshot shows the Jenkins configuration interface for a new job, specifically the 'Source Code Management' tab. The tabs at the top are General, Source Code Management (selected), Build Triggers, Build Environment, Bindings, Build, and Post-build Actions. Blue arrows point to the 'Build Triggers' and 'Post-build Actions' tabs. In the 'Source Code Management' section, the 'Git' radio button is selected. The 'Repositories' section contains a text field for 'Repository URL' with the value 'https://github.com/jenkinsci/jenkins', a dropdown for 'Credentials' set to '- none', and an 'Add' button. Below these are 'Advanced...' and 'Add Repository' buttons. The 'Branches to build' section has a 'Branch Specifier (blank for \'any\')' field with the value '*/2.0' and an 'Add Branch' button. The 'Repository browser' dropdown is set to '(Auto)'. The 'Additional Behaviours' section has a checkbox for 'Clean before checkout' which is checked, and an 'Add' button. A large, semi-transparent watermark 'Build a New Job (2)' is overlaid on the center of the form.

General **Source Code Management** Build Triggers Build Environment Bindings Build Post-build Actions

☐ None
☒ Git

Repositories

Repository URL:

Credentials:

Branches to build

Branch Specifier (blank for 'any'):

Repository browser:

Additional Behaviours

☒ Clean before checkout

Users

☒ Enable project-based security

User/group	Job							Run		SCM
	Delete	Configure	Read	Discover	Build	Workspace	Cancel	Delete	Update	Tag
 xxx	☐	☐	☒	☐	☒	☐	☐	☐	☐	☐
 zzz	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
Anonymous	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

User/group to add:

Add

Roles

**Manage Roles**

Global roles

Role	Overall		Slave		Job					Run			View			SCM	
	Administer	Read	Configure	Delete	Create	Delete	Configure	Read	Build	Workspace	Delete	Update	Artifacts	Create	Delete	Configure	Tag
admin	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
anonymous	☐	☒	☐	☐	☐	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐
job-creator	☐	☒	☐	☐	☒	☐	☐	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐

Role to add

Add

Project roles

Role	Pattern	Job					Run		
		Delete	Configure	Read	Build	Workspace	Delete	Update	Artifacts
		☐	☒	☒	☒	☒	☐	☒	☒
		☒	☒	☒	☒	☒	☒	☒	☒
		☐	☒	☒	☒	☒	☐	☒	☒
Sample Sample.*		☐	☒	☒	☒	☒	☐	☒	☒
		☐	☒	☒	☒	☒	☐	☒	☒

Role to add

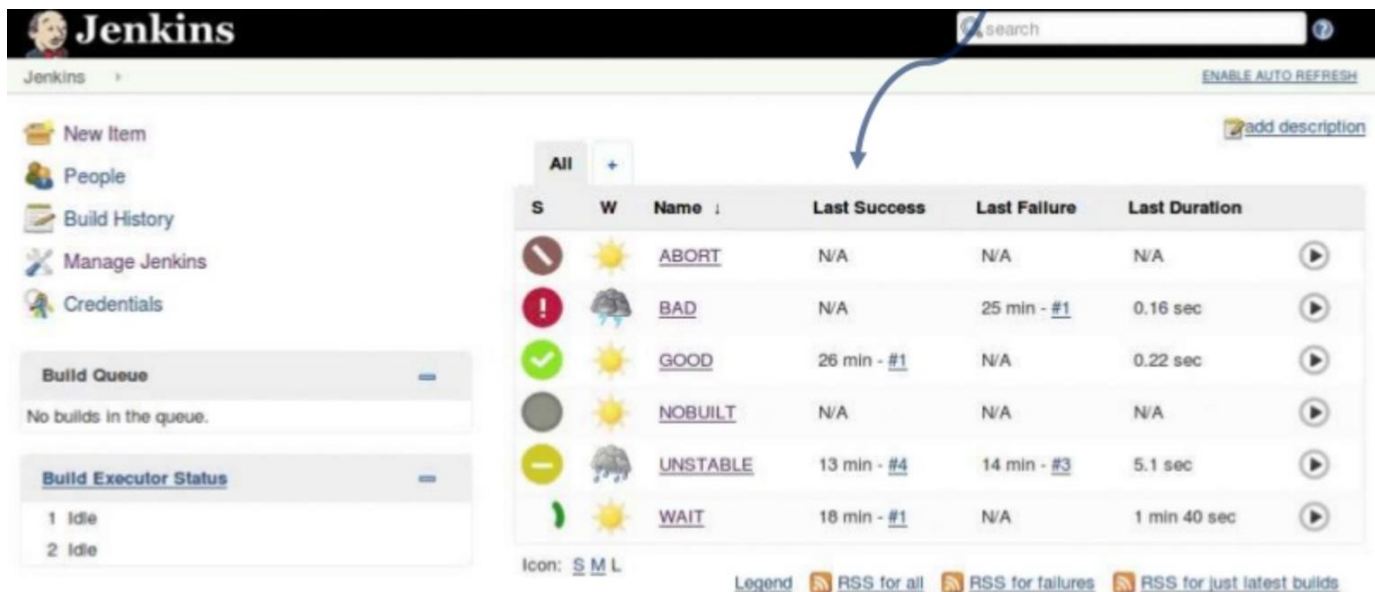
Pattern

Add

Save

BUILD HISTORY

Job Status Summary of Each Recurring Job



The screenshot shows the Jenkins dashboard. On the left is a sidebar with navigation links: New Item, People, Build History, Manage Jenkins, and Credentials. Below these are two expandable sections: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '1 Idle' and '2 Idle'). The main content area features a table of job status summaries. Above the table are filters for 'All' and '+', a search bar, and a link to 'add description'. A blue arrow points from the search bar to the 'Name' column of the table. The table has columns for status icons (S), weather icons (W), job names, last success, last failure, and last duration. Below the table is an 'Icon' legend with links for 'S', 'M', and 'L'. At the bottom, there is a 'Legend' section with RSS links for all jobs, failures, and latest builds.

S	W	Name	Last Success	Last Failure	Last Duration
		ABORT	N/A	N/A	N/A
		BAD	N/A	25 min - #1	0.16 sec
		GOOD	26 min - #1	N/A	0.22 sec
		NOBUILT	N/A	N/A	N/A
		UNSTABLE	13 min - #4	14 min - #3	5.1 sec
		WAIT	18 min - #1	N/A	1 min 40 sec

Icon: [S](#) [M](#) [L](#)

Legend [RSS for all](#) [RSS for failures](#) [RSS for just latest builds](#)

Global Tool Configuration

Jenkins > Global Tool Configuration

[Back to Dashboard](#)

[Manage Jenkins](#)

Global Tool Configuration

Maven Configuration

Default settings provider:

Default global settings provider:

JDK

JDK installations:

List of JDK installations on this system

Git

Git installations

Git	
Name	Default
Path to Git executable	git
☐ Install automatically	

How Many Plugins are There?



Plugins Index

Discover the 1000+ community contributed Jenkins plugins to support building, deploying and automating any project.

Browse Find plugins...

Browse categories	New Plugins	Recently updated	Trending
Platforms	Autocomplete Parameter	Delivery Pipeline	Pipeline: Stage Tags Metadata
User interface	Spring Initializr	Cisco Spark Notifier	Pipeline: GitHub Groovy Libraries
Administration	DotNet as Script	Docker Pipeline	Pipeline: Model API
Source code management	FlexDeploy	Last Changes	Pipeline: Model Definition
Build management	SWAMP	ElasticBox CI	Docker Pipeline
	WSO2 OAuth	bootstrapped-multi-test-results-report	Authentication Tokens API
	Display Console Output	GitHub Pull Request Coverage Status	Docker Commons
	Block Same build	Pipeline Maven Integration	Display URL API
	FTP Rename	JGiven	Credentials Binding

Plugins Management

The screenshot displays the Jenkins Plugin Manager interface. At the top, the Jenkins logo and a search bar are visible. Below the header, there are links for 'Back to Dashboard' and 'Manage Jenkins'. The main section is titled 'Available' and contains a table of plugins. The table has columns for 'Name' and 'Version'. The plugins are categorized into 'Artifact Uploaders' and 'Authentication and User Management'. At the bottom, there are buttons for 'Install without restart' and 'Download now and install after restart'.

Jenkins > Plugin Manager

Filter:

Updates Available Installed Advanced

Install 1

	Name	Version
Artifact Uploaders		
☐	ArtifactDeployer Plugin This plugin makes it possible to copy artifacts to remote locations.	0.27
☐	Artifactory Plugin This plugin allows deploying Maven 2, Maven 3, Ivy and Gradle artifacts and build info to the Artifactory artifacts manager.	2.1.8
☐	Deploy Plugin This plugin takes a war/ear file and deploys that to a running remote application server at the end of a build.	1.9
☐	rhpush Plugin This plugin adds a post build step to push RPMs to Spacewalk or RHN satellite servers. It requires <code>_rhpush_</code> to be installed on the slave.	0.1
☐	S3 Plugin Upload build artifacts to Amazon S3	0.5
☐	SCP plugin This plugin uploads build artifacts to repository sites using SCP (SSH) protocol.	1.8
☐	SFE Plugin Authenticate users and publish build artifacts relying on a Colabnet Source Forge Enterprise Edition (SFE) server.	1.0.4
☒	Vessel plugin This plugin uses the Vessel upload API to allow uploading your iOS/Android files to www.vessel.io	1.0.1
Authentication and User Management		
☐	Active Directory plugin With this plugin, you can configure Jenkins to authenticate the username and the password through Active Directory.	1.33

[Manage Jenkins](#)

[Install without restart](#) [Download now and install after restart](#)

Views

Jenkins search

Jenkins Important View

Recently Chosen Views ENABLE AUTO REFRESH add description

Choose another view

V

Choose another view

Beautiful View

Important View

Nice View

This View has a longer name and several spaces

View of Project X

All Generation Z Important View Nice View View of Project X

S	W	Name ↓	Last Success
		8.02.280-1.2-patch	N/A
		8.02.280P-1.2-patch	N/A
		8.02.281-1.2-patch	N/A
		8.02.281P-1.2-patch	N/A
		8.02.307-1.2-patch	N/A
		8.02.308-1.2-patch	N/A
		8.02.351-1.2-patch	N/A
		8.02.352-1.2-patch	N/A
		8.02.352P-1.2-patch	N/A

Build Queue

No builds in the queue.

Build Executor Status

#	Status
1	Idle
2	Idle

Logger Console Expand

Install a Slave Agent on a New VM

Name	QA
Description	Jenkins slave for deployments
# of executors	1
Remote root directory	C:\Jenkins\
Labels	qa
Usage	Only build jobs with label restrictions matching this node
Launch method	Launch slave agents via Java Web Start
	Tunnel connection through
	JVM options
Availability	Keep this slave on-line as much as possible

Node Properties

☐ Environment variables

☐ Tool Locations

Sometimes This Happens After a Slave Reboot



Slave Test Slave Node

Mark this node temporarily offline

 This node is offline because Jenkins failed to launch the slave agent on it. [See log for more details](#)

Connect slave to Jenkins one of these ways:

-  Launch Launch agent from browser on slave
- Run from slave command line:

```
javaws http://localhost:8080/computer/Test%20Slave%20Node/slave-agent.jnlp
```
- Or if the slave is headless:

```
java -jar slave.jar -jnlpUrl http://localhost:8080/computer/Test%20Slave%20Node/slave-agent.jnlp
```

Created by anonymous user

Projects tied to Test Slave Node

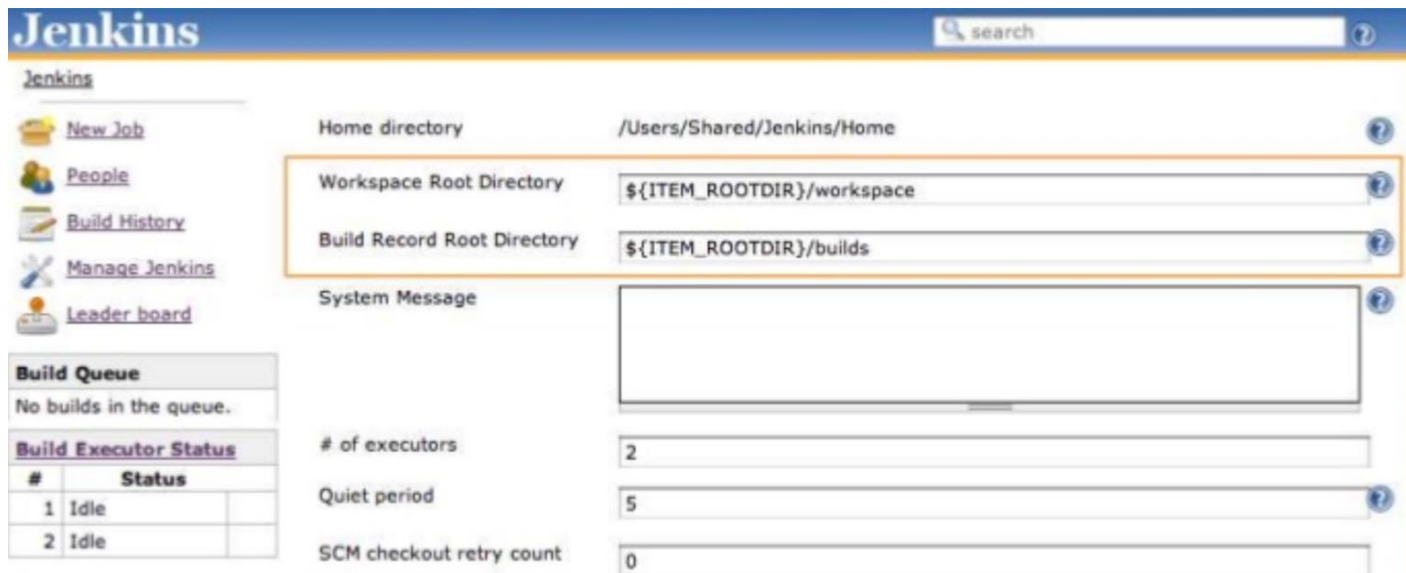
None

Job Console Output

The screenshot shows the Jenkins web interface. The top navigation bar includes 'Back to Project', 'Status', 'Changes', 'Console Output' (selected), 'View as plain text', 'View Build Information', 'Environment Variables', 'Git Build Data', 'HTML Report', 'Test Result', and 'Previous Build'. The 'Console Output' tab is active, displaying a log of build steps. The left sidebar shows 'Timestamps' with options for 'System clock time', 'Elapsed time', and 'None'. The main content area is titled 'Console Output' and shows the following log entries:

```
08:01:59 Started by user Stephen Donner
08:01:59 [EnvInject] - Loading node environment variables.
08:01:59 Building in workspace /var/lib/jenkins/jobs/marketplace.dev/workspace
08:01:59 Wiping out workspace first.
08:01:59 Cloning the remote Git repository
08:01:59 Cloning repository git://gitmirror.mozilla.org/mozilla/marketplace-tests
08:01:59 > git init /var/lib/jenkins/jobs/marketplace.dev/workspace # timeout=10
08:01:59 Fetching upstream changes from git://gitmirror.mozilla.org/mozilla/marketplace-tests
08:01:59 > git --version # timeout=10
08:01:59 > git -c core.askpass=true fetch --tags --progress git://gitmirror.mozilla.org/mozilla/marketplace-tests
+refs/heads/*:refs/remotes/origin/*
08:02:00 > git config remote.origin.url git://gitmirror.mozilla.org/mozilla/marketplace-tests # timeout=10
08:02:00 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
08:02:00 > git config remote.origin.url git://gitmirror.mozilla.org/mozilla/marketplace-tests # timeout=10
08:02:00 Fetching upstream changes from git://gitmirror.mozilla.org/mozilla/marketplace-tests
08:02:00 > git -c core.askpass=true fetch --tags --progress git://gitmirror.mozilla.org/mozilla/marketplace-tests
+refs/heads/*:refs/remotes/origin/*
08:02:00 > git rev-parse origin/master^{commit} # timeout=10
08:02:00 Checking out Revision 3fde0c2a9a3e78ca79873a8af006cd2405765a20 (origin/master)
08:02:00 > git config core.sparsecheckout # timeout=10
08:02:00 > git checkout -f 3fde0c2a9a3e78ca79873a8af006cd2405765a20
08:02:00 > git rev-list 3fde0c2a9a3e78ca79873a8af006cd2405765a20 # timeout=10
08:02:00 provisioning config files...
08:02:00 copy managed file [marketplace.yaml] to file:/tmp/config705907656409285219/tmp
08:02:00 [EnvInject] - Executing scripts and injecting environment variables after the SCM step.
08:02:00 [EnvInject] - Injecting as environment variables the properties content
08:02:00 PATH=$WORKSPACE/.env/bin:/usr/local/bin:/usr/sbin:/bin:/usr/bin
08:02:00
08:02:00 [EnvInject] - Variables injected successfully.
08:02:00 executing script 'Create virtual environment'
08:02:00 [workspace] $ /bin/bash -ex /tmp/build_step_template7831564549360508090.sh
08:02:00 + virtualenv .env
08:02:00 New python executable in .env/bin/python2.7
08:02:00 Also creating executable in .env/bin/python
08:02:00 Installing setuptools, pip...done.
08:02:00 executing script 'Install infrastructure requirements'
08:02:02 [workspace] $ /bin/bash -ex /tmp/build_step_template30291506844090501.sh
08:02:02 + pip install -r https://raw.githubusercontent.com/mozilla/webqa-infrastructure-requirements/master
/requirements.txt
08:02:03 DEPRECATION: --download-cache has been deprecated and will be removed in the future. Pip now
automatically uses and configures its cache.
08:02:04 Collecting pytest-rerunfailures==0.9.3 (from -r https://raw.githubusercontent.com/mozilla/webqa-
infrastructure-requirements/master/requirements.txt (line 4))
08:02:04 Using cached pytest-rerunfailures-0.9.3.tar.gz
08:02:04 /var/lib/jenkins/jobs/marketplace.dev/workspace/.env/lib/python2.7/site-packages/setuptools
/dist.py:283: UserWarning: The version specified requires normalization, consider using '0.9.3'.
```

Each Job has its own space



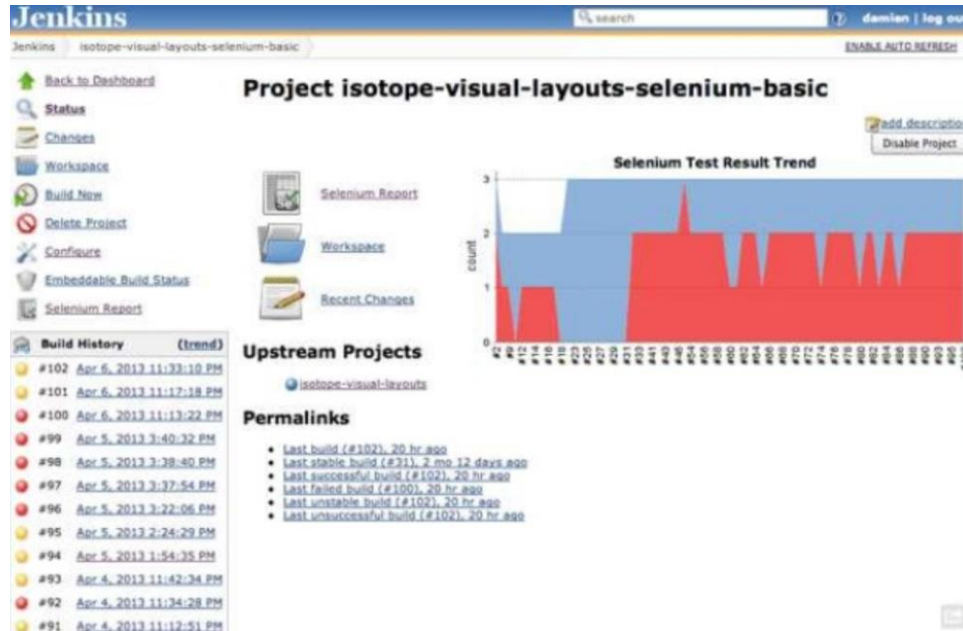
The screenshot shows the Jenkins configuration interface for a new job. The left sidebar contains navigation links: [Jenkins](#), [New Job](#), [People](#), [Build History](#), [Manage Jenkins](#), and [Leader board](#). Below these are sections for the **Build Queue** (showing no builds) and **Build Executor Status** (showing two idle executors).

The main configuration area on the right includes the following settings:

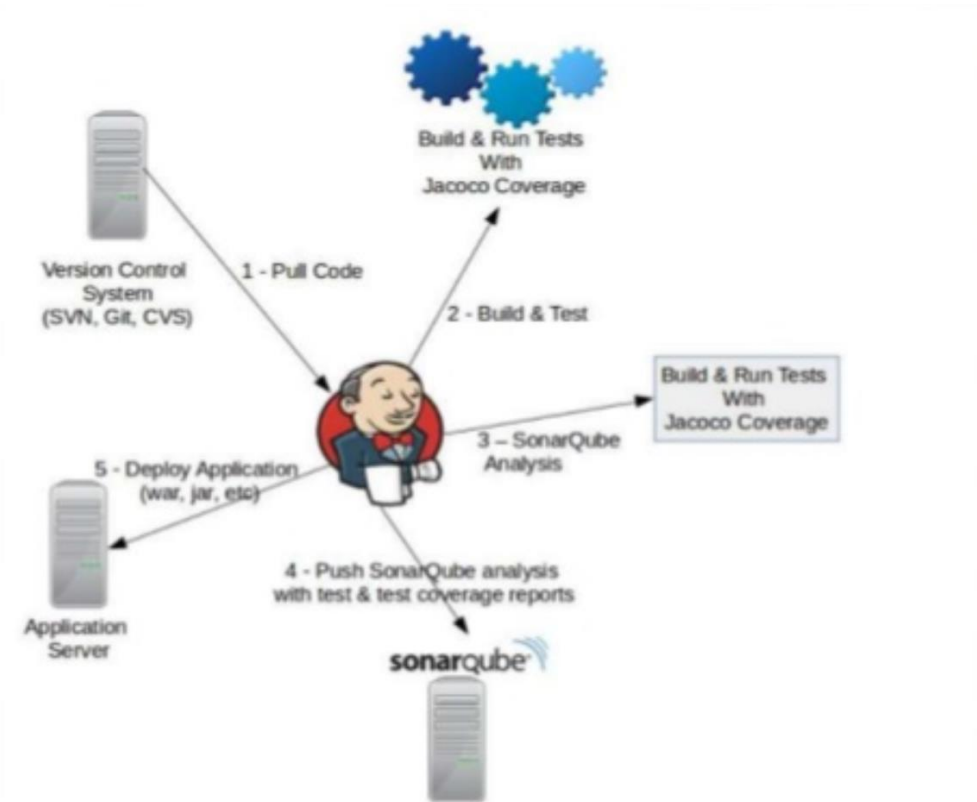
- Home directory:** /Users/Shared/Jenkins/Home
- Workspace Root Directory:** \${ITEM_ROOTDIR}/workspace (highlighted in an orange box)
- Build Record Root Directory:** \${ITEM_ROOTDIR}/builds (highlighted in an orange box)
- System Message:** (empty text area)
- # of executors:** 2
- Quiet period:** 5
- SCM checkout retry count:** 0

Each configuration field has a help icon (question mark) to its right.

Jenkins and Selenium Testing Tool (software) Work well together



Example of How Jenkins Enables CI/CD Pipelines



Multi-step Python Code Job on Jenkins

Build

 **Execute Python script**

Script `execfile("jobscript.py")`

[See the list of available environment variables](#)

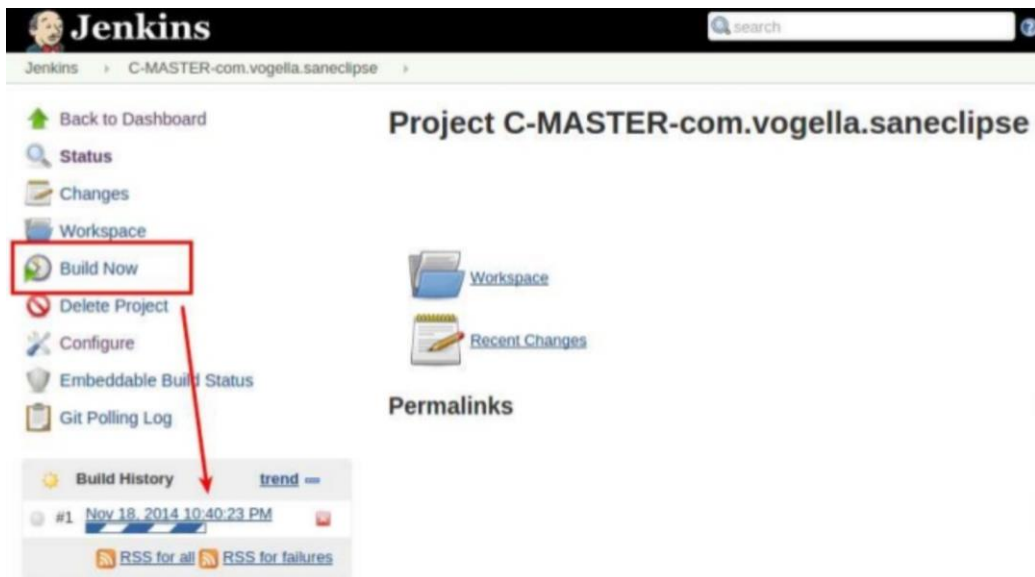
Add build step ▼

Post-build Actions

Add post-build action ▼

Save **Apply**

Kick off a Build Right Now





Jenkins & Continuous Integration

What is it?

- Continuous integration server - **detects** changes in Subversion, **performs** tasks, repeatedly. (Build, Test, Deploy, Test, Package)
- Seven years old approx
- 47000+ installations
- 600+ plugins
- A fork of the original Hudson project

Why do we need it?

- To *integrate more frequently*, detect *errors* quicker, improve *quality* and reduce *cost*
- Co-ordinate the running of tasks as part of workflows.
- Compile, test and package, deploy, script, verify, build vm's

Jenkins Alternatives

TeamCity - allows for personal builds, pre- tested commits

Bamboo - Atlassians CI server, good, but somewhat inflexible

Cruise Control - bleurgh, old, behind the times

Cron and bash...

Terminology

- Job - a unit of work for a project
- View - user defined collection of jobs or a workflow
- Master - the central Jenkins master, does job scheduling
- Slave - executes one or more jobs within slots (executors)
- Workspace - the working area where a job is carried out
- Plugin - user defined collection of jobs or a workflow



jenkins-modslave
Windows XP
Java JDK 6



cismaster

Slaves connect to cismaster over JNLP



cistrwin1
Windows 7
R (2.8.1 and upwards)



cisjava1
Centos 5.9
Java JDK 6



cisjava2
Centos 5.9
Java JDK 6

javaunixslaves



cisjavawin
Windows 7
Java JDK 6

javawindowsslaves



cislave-w7net4
Windows 7
.NET 4



cistr1
Centos 5.9
R (2.8.1 and upwards)

Jenkins Main User Interface

Jenkins search services | log out

Jenkins - Glance Views

[New Job](#) [People](#) [Build History](#) [Edit View](#) [Delete View](#) [Project Relationship](#) [Check File Fingerprint](#) [Manage Jenkins](#) [Query and Trigger Agent Patches](#) [My Views](#) [Job Config History](#)

Build Queue
No builds in this queue.

Build Executor Status

#	Slave	Build	Status
1	idle		
2	idle	build (offline)	
1	idle	build1	
		build2 (offline)	
		build3 (offline)	
		build4 (offline)	
		gl	

Views

W	Name	Last Success	Last Failure	Last Duration
🟢	glance	15 hr (#526)	5 days 15 hr (#5277)	2 min 3 sec
🟢	glance-merge	15 hr (#119)	1 mo 2 days (#79)	3 sec
🟢	glance-pool	15 hr (#209)	16 days (#209)	6.5 sec
🟢	keystone	12 hr (#400)	13 hr (#500)	1 min 45 sec
🟢	keystone-merge	12 hr (#500)	6 days 15 hr (#567)	4.6 sec
🟢	keystone-pool	12 hr (#139)	6 days 15 hr (#135)	9 sec
🟢	keystone-xyld	12 hr (#400)	6 days 15 hr (#303)	22 sec
🟢	nova	3 hr 44 min (#12729)	19 hr (#12731)	8 min 19 sec
🟢	nova-merge	3 hr 44 min (#215)	21 hr (#154)	1 min 0 sec
🟢	nova-pool	3 hr 44 min (#1508)	21 hr (#1567)	1 min 9 sec
🟢	quantum	9 hr 37 min (#60)	9 days 9 hr (#60)	9.1 sec
🟢	quantum-pool	9 hr 37 min (#50)	N/A	4.4 sec
🟡	quantum-xyld	9 hr 35 min (#31)	N/A	20 sec
🟢	swift	21 hr (#119340)	1 mo 17 days (#119320)	14 sec
🟢	swift-merge	21 hr (#150)	N/A	3.3 sec

List of available jobs

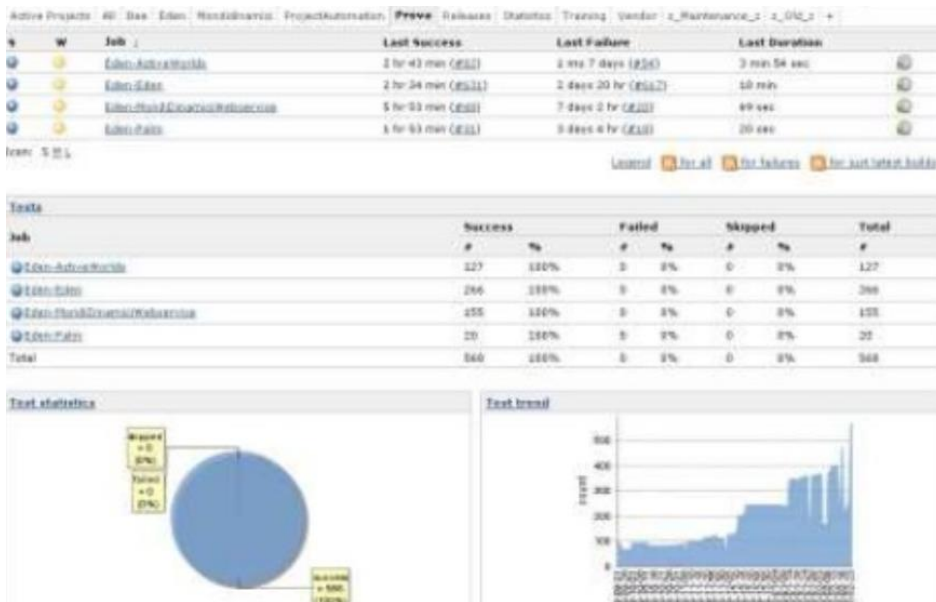
List of jobs being executed

Custom Jenkins Views

You can build your own views of Jenkins jobs

Jobs of interest that you're working on

You can also build custom views with the dashboard plugin



Investigating a Failure

Look at the build log in detail

Try executing the job on the slave (via SSH)

Permissions are a common issue

Jenkins Plugins

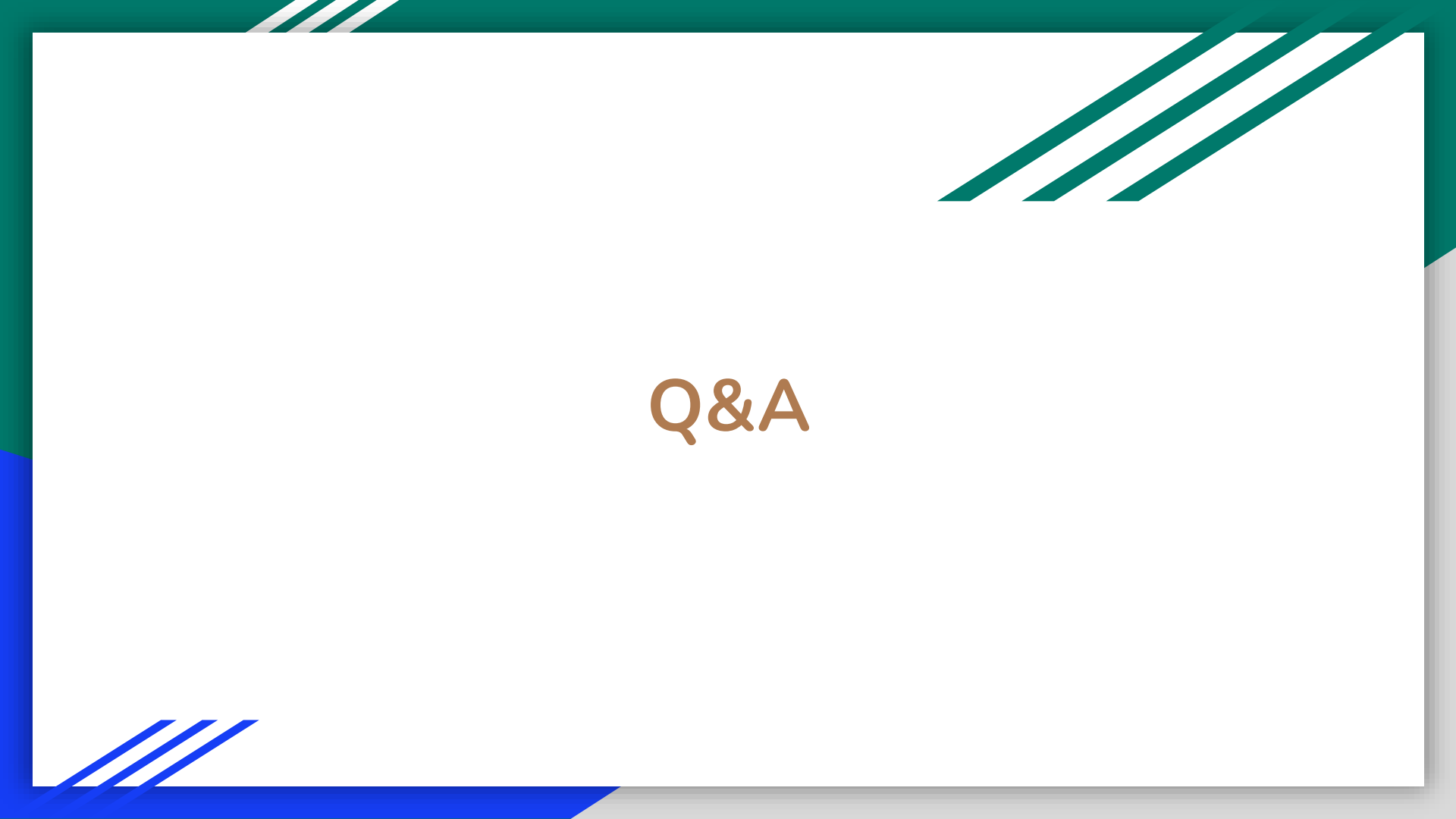
- Subversion - polls for changes to build
- Sonar - publishes metrics on builds
- Parameterised triggers - trigger builds on conditions
- Copy Artifact - copy artifact for completed builds
- Workflow - setup build workflows
- JClouds - provisions environments

Jenkins Tips

When a job fails - **claim the failure** so people know what's going on. Send an email out

Use parameterised builds - **don't hardcode paths** to R, Java etc... think generic

Don't install dependencies on Jenkins build servers. Build your own environment!



Q&A